# Systemadministration – Virtualisering och molntjänster

*Containrar*

## Laboration 4

### Poäng och betygsättning

Laborationen betygsätts U/G utan poängsättning. Deadline är en vecka från labbstart.

### Utförande

Uppgiften utförs i grupper om två till tre studenter. Tidigare serverkitt delas mellan grupperna. Studenterna ansvarar för gruppindelningen.

Misstänkt fusk, plagiat och trakasserier anmäls till *Disciplin- och avskiljandenämnden* (DAN).

### Syfte

Studenten skall visa färdighet i planering och uppsättning av virtualiseringslösningar för servrar, nätverk och applikationer. Studenten skall visa förmåga att hantera centraliserade lagringslösningar och drift av mindre datacenter. Studenten skall visa förmåga att utforma IT-system som bättre utnyttjar befintliga resurser

### Handledning

Handledning sker under ordinarie laborationstillfällen.

### Redovisning

Laborationen redovisas i labbsal för närvarande labbhandledare.

# 1 Labbmiljö

Hittills har tjänster installerats på operativsystem som i sin tur körs på virtuella maskiner. Nu ska istället tjänster tillhandahållas med *container*-baserad virtualisering.

I *laboration 3* utforskades webbvirtualisering. I denna laboration ska *backend*-webbservrarna ersättas med *container* virtualiserade webbservrar. Se därför till att lastbalanserarna (HAProxy) är igång och fungerar innan du fortsätter.

Skapa därefter två nya Debian-maskiner ifrån din template

> Tips!
> *Båda maskinerna behöver samma konfiguration. Konfigurera därför de båda parallellt, och dela upp arbetet inom gruppen, för att spara lite tid.*

# 2 Docker Installation

Installera *Docker* på de båda maskinerna (se utdrag 1).

```
# install docker
sudo apt install docker.io

# give user nics permission to use docker commands
usermod -aG docker nics
logout
```

Utdrag 1: *Installation och tilldelning av rättigheter.*

# 3 Test av Docker

Validera att *Docker* är korrekt installerat med en test-*container*, använd utdrag 2 som referens.

Kommandot **docker run hello-world** kommer att ladda ned en existerande *container-image* ifrån https://hub.docker.com, skapar och sedan kör din *container*.

```
docker run hello-world
docker image ls
docker container ls -a
```

Utdrag 2: *Docker test-container och kommandon.*

Notera att *containern* inte längre körs. Till skillnad från virtuella maskiner så avslutas *containrar* när den har exekverat sin sista instruktion. *Dockerfilen* som definerar vad *containern* ska göra innehåller enbart tre rader av instruktioner. Vilka instruerar *Docker* att skapa en ny *container* som printar ut ett meddelande till konsolen (se utdrag 3).

```
FROM scratch
COPY hello /
CMD ["/hello"]
```

Utdrag 3: *hello-world Dockerfile.*

Om *Docker* ser ut att fungera kan du ta bort din test *container*. Använd **docker container rm <container-id>** för att ta bort en specifik *container*, eller **docker container prune** för att ta bort samtliga avstängda *containrar*. Ta även bort alla oanvända *container-images* med **docker image prune**.

*Dockers* olika kommandon blir snabbt långa, ett tips är därför att använda **--help** flaggan för att visa möjliga alternativ. Prova till exempel efter kommandot **docker container**. Många vanliga kommandon har även kortare alternativ, dock kan dessa lätt blandas ihop – som med exempelvis **docker images** och **docker image**.

# 4 Docker Images

Ett vanligt sätt att använda *containrar* är genom att utnyttja redan existerande *images*. Många tjänster finns tillgängliga som färdiga *images* som är skapade av communityt eller av utvecklarna själva. Kika gärna runt på https://hub.docker.com för att se vad som finns tillgängligt. Att installera en servertjänst med *Docker* är då ofta precis lika smidigt som att installera **hello-world**.

För att testa en vanligt servertjänst ska webbservern *NGINX* installeras, se utdrag 4. Börja därför med att ladda ned en *NGINX container-image*. Starta sedan en ny *container* med *imagen* där *Debians* port 80 även knyts till *containerns*.

Kontrollera sedan att webbservern fungerar genom att surfa till din VM.

```
docker image pull nginx:stable
docker run --name nginx -p 80:80 nginx:stable
```

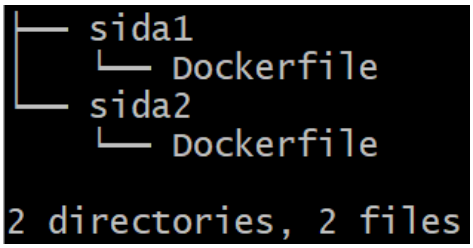
Utdrag 4: *NGINX container.*

När du har kontrollerat att webbservern fungerar, kan du avsluta *containern* med **Ctrl-C**.

# 5 Dockerfile

Vi ska nu skapa en liknande miljö som gjordes i *laboration 3*. Där två hemsidor körs på samma maskin, och som tillhandahålls redundant med hjälp av lastbalansering.

Börja med att skapa två kataloger i **/srv/**, en för varje hemsida, skapa sedan en **Dockerfile** i varje katalog (använd figur 1 som referens).




Figur 1: *Docker filstruktur.*

> Obs!
> *Dockerfilen behöver heta exakt "**Dockerfile**".*

Det finns sedan två alternativ i hur *containrarna* ska konfigureras. Antingen kan man använda ett par förberedda hemsidor eller så kan man skapa sina egna.

## 5.1 Dockerfile: Med förberedd hemsida

För den första hemsidan skriv in följande i **Dockerfile**:

```
FROM nginx:stable

RUN curl https://raw.githubusercontent.com/hakanhammarin/dockerfiles/
    main/node01.html -o /usr/share/nginx/html/index.html

EXPOSE 80
```

Utdrag 5: *Dockerfile för hemsida1.*

För den andra hemsidan skriv in följande i **Dockerfile**:

```
FROM nginx:stable

RUN curl https://raw.githubusercontent.com/hakanhammarin/dockerfiles/
    main/node02.html -o /usr/share/nginx/html/index.html

EXPOSE 80
```

Utdrag 6: *Dockerfile för hemsida2.*

## 5.2 Dockerfile: Med egen hemsida

Alternativt om du vill skapa din egen hemsida kan du göra som i utdrag 7.

```
mkdir /srv/sida1/html
editor /srv/sida1/html/index.html
```

Utdrag 7: *Kommandon för egenskapad hemsida.*

I sådana fall behöver även din **Dockerfile** ändras till:

```
FROM nginx:stable

COPY ./html/ /usr/share/nginx/html/

EXPOSE 80
```

Utdrag 8: *Alternativ Dockerfile.*

### 5.3 Dockerfile: Skapa containrar

När **Dockerfilerna** är definerade för de båda hemsidorna kan vi generera *container-images* (se utdrag 9).

```
docker image build --no-cache -t sida1:1.0 /srv/sida1
docker image build --no-cache -t sida2:1.0 /srv/sida2
```

Utdrag 9: *Skapande av container-image för hemsida1 & hemsida2.*

Starta sedan de två *containrarna*, se dock till att de körs i bakgrunden den här gången (se utdrag 10).

```
docker container run -d --name sida1 -p 8081:80 sida1:1.0
docker container run -d --name sida2 -p 8082:80 sida2:1.0
```

Utdrag 10: *Containrar för hemsida1 & hemsida2.*

Se till att dina båda *container-images* och *containrar* har skapats korrekt (se utdrag 11). Kontrollera sedan att de båda hemsidorna fungerar genom att surfa till din VM. Observera att portarna skiljer sig åt.

```
docker image ls
docker container ls
docker stats
docker logs sida1 -f
```

Utdrag 11: *Kontrollera containrar.*

Tips!

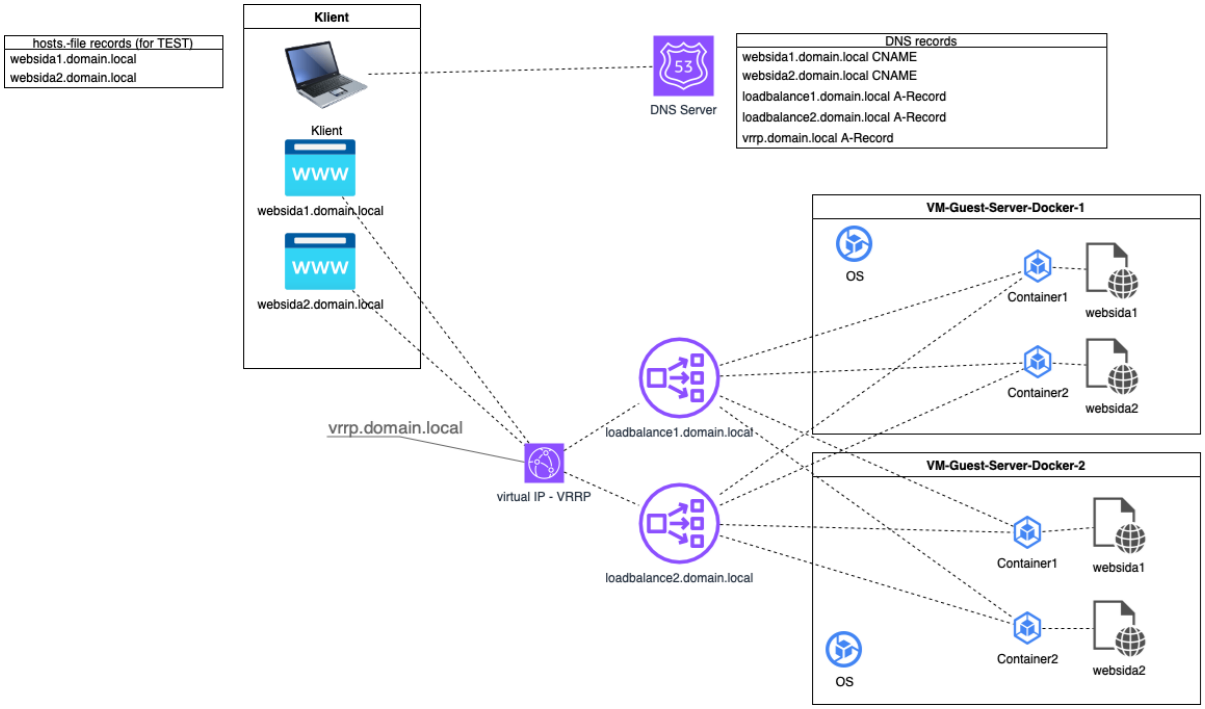
*För att se en containers loggar kan kommandot **docker logs -f** användas (motsvarar **tail -f /var/log/file**).*

## 6 High Availability Groups

Se till att dina Docker-maskiner inte körs på samma fysiska server med hjälp utav *HA groups*. Du kan läsa mer om HA groups här: https://pve.proxmox.com/pve-docs/pve-admin-guide.html#ha_manager_groups

# 7 HAproxy

Nu när vi ska ersätta webbservrarna med containrar behöver miljön uppdateras (se figur 2).




Figur 2: *Ny container-baserad arkitektur.*

Justera konfigurationen i HAproxy så att de nya containrarna används som *backend*. Se även till att HAproxy använder rätt *backend*-par för rätt *virtual host*/*host header*. Använd utdrag 12 som referens.

```
frontend http_haproxy
    bind *:80
    mode http
    use_backend webserver_1 if { req.hdr(host) -i websida1.domain.local
        }
    use_backend webserver_2 if { req.hdr(host) -i websida2.domain.local
        }
```

Utdrag 12: *HAproxy konfiguration.*

**Tips!**

*Notera även att webbsidorna använder olika portar*

**Fråga 1**

Redovisa i labbsal att lastbalansering av de container-baserade hemsidorna fungerar.